

Contents

Where and How to Use Markdown	1
Quick choice: where should I write Markdown?	2
1. Using Markdown in VS Code	2
Step 1 - Install VS Code	2
Step 2 - Create a Markdown file	2
Step 3 - Open the Markdown preview	3
macOS	3
Windows / Linux	3
Step 4 - Edit source and watch the preview	3
Step 5 - Add an image	4
Step 6 - Add math when the renderer supports it	4
Optional VS Code extensions	4
2. Using Markdown directly on GitHub	4
Edit a README without Git or VS Code	4
Good uses of GitHub Markdown	5
A privacy reminder	5
3. Using HackMD in the browser	5
Beginner workflow	5
4. Using StackEdit in the browser	6
Beginner workflow	6
5. Markdown in Jupyter Notebook or Google Colab	7
6. Markdown with Quarto	7
7. Which tool should a non-technical PhD student choose?	8
Use VS Code if	8
Use GitHub's web editor if	8
Use HackMD if	8
Use StackEdit if	8
Use Jupyter / Colab if	8
Use Quarto if	8
8. Important: Markdown looks slightly different across tools	8
9. Moving the same file between VS Code, GitHub, and web editors	9
10. Exporting Markdown to PDF	9
Simplest option	9
Reproducible academic option: Pandoc	9
Quarto	10
11. Five-minute practice exercise	10
12. Recommended next file in this repository	11

Where and How to Use Markdown

A beginner-friendly setup guide for PhD students

Markdown is much easier to start than LaTeX because **you usually do not need a compiler or special installation**. A Markdown file is simply a plain-text file ending in `.md`.

If you can create a file called `notes.md`, type text into it, and open a preview, you can use Markdown.

Quick choice: where should I write Markdown?

Your situation	Recommended tool	Install anything?	Best for
"I already use VS Code."	VS Code	VS Code only	Local notes, GitHub projects, research documentation
"I want nothing installed."	GitHub web editor	No	READMEs and repository documentation
"I want a Google-Docs-like collaborative Markdown editor."	HackMD	No	Shared notes, meetings, collaborative drafts
"I want a browser editor with a live preview."	StackEdit	No	Quick Markdown writing and previewing
"My notes live next to code and data analysis."	Jupyter / Colab / Quarto	Depends	Computational research notebooks and reproducible documents

Beginner recommendation:

- If you already downloaded this repository, use **VS Code**.
- If you do not want to install anything, use the **GitHub web editor**, **HackMD**, or **StackEdit**.
- If you work with R/Python notebooks, use Markdown cells in **Jupyter/Colab**, or move to **Quarto** later.

Markdown is portable. The same `.md` file can usually be opened in VS Code, GitHub, HackMD, StackEdit, Obsidian, and many other tools. However, advanced features can differ because the tools use different Markdown "flavors."

1. Using Markdown in VS Code

VS Code has built-in Markdown support. For basic Markdown writing and previewing, **you do not need a Markdown extension**.

Official documentation: <https://code.visualstudio.com/Docs/languages/markdown>

Step 1 - Install VS Code

Download VS Code from:

<https://code.visualstudio.com/>

If you already installed VS Code for LaTeX, you are ready.

Step 2 - Create a Markdown file

In VS Code:

1. choose **File -> New Text File**;
2. save it as something ending in `.md`, for example:

paper_notes.md

3. type:

Paper Notes

Main question

What is the mechanism behind the result?

Tasks

- [] Re-read Section 3
- [] Check the appendix
- [] Reproduce Figure 2

That is already a valid Markdown document.

Step 3 - Open the Markdown preview

VS Code can show the rendered document instead of the raw #, **, and backticks.

macOS

Open preview:

Shift + Command + V

Open preview beside the source:

Command + K, then V

Windows / Linux

Open preview:

Ctrl + Shift + V

Open preview beside the source:

Ctrl + K, then V

The side-by-side view is particularly good for beginners:

Markdown source	Rendered preview
-----	-----
# Results	Results

Finding: ...	Finding: ...

You can also open the Command Palette and search for:

Markdown: Open Preview

Markdown: Open Preview to the Side

Step 4 - Edit source and watch the preview

A simple workflow is:

1. keep the .md file on the left;
2. keep the preview on the right;
3. make one change;
4. save;
5. check the preview.

Unlike LaTeX, basic Markdown does **not** require a compile cycle.

Step 5 - Add an image

Suppose your project looks like:

```
project/  
  notes.md  
  figures/  
    result.png
```

In `notes.md`, write:

```
![Main result](figures/result.png)
```

Use **relative paths** like this when possible. They are much easier to move between VS Code and GitHub than paths such as `/Users/name/Desktop/...`

Step 6 - Add math when the renderer supports it

Many research-oriented Markdown tools support LaTeX-style math.

Inline:

The parameter is β .

Display:

```
$$  
y_i = x_i'\beta + \varepsilon_i,  
\quad i=1,\ldots,n.  
$$
```

Math support varies across Markdown tools, so always check the preview in the tool where the document will be read.

Optional VS Code extensions

You do not need these to start. Add them only if you want the extra features.

- **markdownlint** - warns about inconsistent Markdown style.
- **Markdown All in One** - adds shortcuts and convenience features.

For a complete beginner, the built-in VS Code preview is enough.

2. Using Markdown directly on GitHub

GitHub renders Markdown in repository `README.md` files and throughout its web interface. GitHub uses **GitHub Flavored Markdown (GFM)**, which includes features such as tables, task lists, and strikethrough.

Official guide: <https://docs.github.com/en/get-started/writing-on-github/getting-started-with-writing-and-formatting-on-github/basic-writing-and-formatting-syntax>

Edit a README without Git or VS Code

You can edit Markdown entirely in the browser.

1. Open a repository on GitHub.
2. Open `README.md` or another `.md` file.
3. Click the edit/pencil control.
4. Edit the Markdown source.

5. Use the preview option to inspect the rendered result.
6. Commit/save the change.

For a new repository, a simple README can begin with:

Replication Files

This repository contains the code and documentation for the project.

Folder structure

- ``data/`` - input data
- ``code/`` - analysis code
- ``output/`` - generated tables and figures

Reproduction

1. Run ``01_clean.R``.
2. Run ``02_analysis.R``.
3. Check ``output/``.

Good uses of GitHub Markdown

- repository README.md files;
- replication instructions;
- project documentation;
- issue descriptions;
- pull-request descriptions;
- public teaching materials.

A privacy reminder

Do not put restricted data, confidential participant information, private API keys, or material prohibited by a data-use agreement into a public GitHub repository.

3. Using HackMD in the browser

HackMD is a browser-based Markdown workspace designed around writing, sharing, and real-time collaboration.

Website: <https://hackmd.io/>

It is useful when you want something closer to a collaborative shared document while still keeping the underlying content in Markdown.

Beginner workflow

1. Open HackMD and create/sign in to an account.
2. Create a new note.
3. Type Markdown in the editor.
4. Use the rendered view/preview to check formatting.
5. Share the note with collaborators if needed.
6. Check sharing permissions before adding unpublished or sensitive material.

A coauthor-meeting note could look like:

Coauthor Meeting - 2026-09-26

Decisions

- Keep the main sample definition.
- Move robustness results to the appendix.

Questions

- Why does the estimate change in the rural subsample?

Next actions

- [] Update Table 2
- [] Re-run robustness checks
- [] Draft response to referee comment 4

HackMD is especially convenient for:

- meeting notes;
 - shared research notes;
 - reading groups;
 - collaborative outlines;
 - lightweight documentation.
-

4. Using StackEdit in the browser

StackEdit is an in-browser Markdown editor with a live preview.

Website: <https://stackedit.io/>

Its official site describes support for Markdown flavors including GitHub Flavored Markdown and CommonMark, live preview, LaTeX-style mathematical expressions, and synchronization options with services such as Google Drive, Dropbox, and GitHub.

Beginner workflow

1. Open StackEdit.
2. Start a new document.
3. Type Markdown in the source/editor pane.
4. Watch the rendered preview.
5. Export or synchronize the document if needed.

StackEdit is useful when you want to practice Markdown in a browser without setting up a local project.

Example:

Seminar Notes

Speaker: Jane Doe
Topic: Labor market adjustment

Main contribution

The paper studies ...

Equation I want to revisit

\$\$

```
y_i = \alpha + \beta x_i + \varepsilon_i.  
$$
```

Questions

1. How is the sample selected?
 2. What is the identifying variation?
-

5. Markdown in Jupyter Notebook or Google Colab

If you analyze data in Python, Julia, or R notebooks, you can combine **code cells** with **Markdown cells**.

A Markdown cell can contain:

Results

The coefficient is positive in the baseline sample.

```
$$  
\hat{\beta} = 0.12.  
$$
```

The next cell reproduces the figure.

Then the next notebook cell can contain the actual Python/R code.

This is useful for:

- exploratory analysis;
- computational notes;
- teaching notebooks;
- documenting why a code block exists;
- keeping equations and interpretation next to results.

A notebook is not the same as a plain `.md` file, but Markdown syntax is used inside its text cells.

6. Markdown with Quarto

Quarto is useful when you eventually want Markdown-like source plus:

- citations;
- cross-references;
- equations;
- executable R/Python/Julia code;
- HTML, PDF, or Word output.

A Quarto source file usually ends in `.qmd` rather than `.md`.

Example:

```
---  
title: "Research Note"  
format: html  
---
```

Results

The analysis is summarized below.

You can edit Quarto in VS Code or RStudio. It is a good next step after ordinary Markdown, but you do not need it to learn Markdown.

7. Which tool should a non-technical PhD student choose?

Use VS Code if ...

- you already use it for LaTeX;
- you want your notes stored as local files;
- you want one folder containing code, Markdown, figures, and LaTeX;
- you plan to use GitHub later.

Use GitHub's web editor if ...

- you mainly need to edit a README;
- you want to contribute a small documentation change;
- you do not want to install an editor.

Use HackMD if ...

- several people need to edit a note together;
- you want browser-based collaboration;
- you want a shared seminar/coauthor document.

Use StackEdit if ...

- you want a browser editor with a live preview;
- you are practicing Markdown;
- you want to work without configuring VS Code.

Use Jupyter / Colab if ...

- prose belongs directly beside executable code;
- the document is part of data analysis rather than a standalone note.

Use Quarto if ...

- you want reproducible academic documents from Markdown-like source;
 - you need citations, cross-references, code execution, and multiple output formats.
-

8. Important: Markdown looks slightly different across tools

Do not assume every feature works everywhere.

The following are broadly portable:

Heading

bold

italic

- bullet

1. numbered item

`[link](https://example.com)`

``inline code``

These may vary by renderer:

- mathematical expressions;
- footnotes;
- task lists;
- callouts/admonitions;
- citations;
- automatic cross-references;
- diagrams;
- YAML metadata.

Rule: write using the common Markdown core when portability matters, and use advanced features only when you know which renderer will display the file.

9. Moving the same file between VS Code, GitHub, and web editors

Because Markdown is plain text, the basic workflow is simple:

```
notes.md
|
+--> edit locally in VS Code
+--> upload/commit to GitHub
+--> open/copy into HackMD
+--> open/copy into StackEdit
```

For maximum portability:

1. keep images in the project folder;
 2. use relative image links;
 3. avoid tool-specific syntax unless necessary;
 4. keep filenames simple;
 5. save the source as UTF-8;
 6. preview the file on the platform where readers will view it.
-

10. Exporting Markdown to PDF

Markdown itself is only source text. PDF export depends on the tool.

Simplest option

If your editor provides a print/export view, use it for a quick PDF.

Reproducible academic option: Pandoc

If Pandoc and a LaTeX distribution such as TinyTeX are installed:

```
pandoc notes.md -o notes.pdf
```

This is useful when you want the same Markdown source to generate a repeatable PDF.

Quarto

For a Quarto file:

```
quarto render notes.qmd --to pdf
```

For beginners, PDF export is optional. Learn ordinary Markdown writing and previewing first.

11. Five-minute practice exercise

Create a file named:

```
my_first_markdown_note.md
```

Paste:

```
# My First Research Note
```

```
## Question
```

What am I trying to understand?

```
## Notes
```

```
**Main idea:** Write one sentence here.
```

- Evidence 1
- Evidence 2
- Evidence 3

```
## Tasks
```

- [x] Create this file
- [] Add one paper citation/link
- [] Add one equation
- [] Add one image

```
## Code or command
```

Run ``python analysis.py`` after cleaning the data.

```
## Equation
```

```
$$  
y_i = x_i'\beta + \varepsilon_i,  
\quad i=1,\ldots,n.  
$$
```

Then open the preview in VS Code or paste the same source into a browser Markdown editor.

You now know enough Markdown to create useful research notes.

12. Recommended next file in this repository

After completing this guide, read:

`04_Markdown/PhD_Markdown_Beginner_Guide.md`

Use its PDF version if you want to read the guide without editing the source.